

Heavy Equipment Selling Price Prediction

Machine Learning Project Report

Kaggle Competition: Heavy Equipment Selling Price Prediction Challenge

Roll No: 23f2003210

1. Abstract

This report documents an end-to-end classical machine learning solution to the Heavy Equipment Selling Price Prediction Challenge, a Kaggle competition in which each row describes a used heavy-equipment transaction (auction/resale of excavators, wheel loaders, backhoes and similar machinery) via a mix of numeric usage fields, low/medium-cardinality categorical fields, high-cardinality identifier/specification fields, and a large block of largely-structural-missing technical attachment columns. Submissions are scored with Root Mean Squared Logarithmic Error (RMSLE), so the target TargetValue (the sale price in USD) is modelled in log1p space throughout.

Four regression families were compared under an identical, leakage-safe preprocessing pipeline: a Random Forest baseline, and three gradient-boosting libraries explicitly permitted by the competition rules — XGBoost, LightGBM and CatBoost. After individual RandomizedSearchCV tuning against a custom RMSLE scorer, a validation-tuned weighted-average ensemble of the three tuned boosters improved on every individual model, reaching a validation RMSLE of 0.2088 versus 0.2105 for the best single tuned model (XGBoost). A subsequent, higher-leverage revision replaced the single 80/20 split with 5-fold cross-validation, swapped LightGBM (found to be redundant with XGBoost) for a CPU-native HistGradientBoostingRegressor, gave CatBoost its categorical columns natively instead of ordinal codes, and blended the three folds' out-of-fold predictions with non-negative least squares (NNLS) — lowering the out-of-fold RMSLE further to 0.20217, which was used to generate the final Kaggle submission.

The project's main learning was that careful, leakage-aware feature engineering (structural-missingness flags, frequency encoding of high-cardinality IDs, K-fold target encoding, and an explicit equipment-age feature) mattered at least as much as model choice, and that a lightweight, validation/OOF-driven ensemble of diverse tree-based models reliably outperforms any single model on this kind of wide, mixed-type tabular data.

2. Introduction

2.1 Business Problem

Heavy industrial equipment (excavators, wheel loaders, backhoes, etc.) is routinely bought and sold on the secondary (auction/resale) market. Unlike new equipment, the price of a used machine depends on many factors — its age, accumulated operating hours, technical configuration, region of sale, and specific attachments/specifications. For a platform that facilitates these transactions (auction houses, resellers, financing companies), being able to automatically estimate a fair selling price is valuable: it helps sellers set realistic reserve prices, helps buyers judge whether a listed price is fair, and helps the platform flag mispriced listings, detect anomalies, and build financing/insurance products around an asset's estimated value.

Given historical transaction records (train.csv) containing the technical specification, usage, geography and sale details of previously sold machines along with their observed sale price, the task is to predict the selling price (TargetValue) of equipment in test.csv whose price is unknown.

2.2 Target Variable — TargetValue

TargetValue is the financial amount (in USD) at which a piece of equipment was sold. It is a continuous, strictly positive, right-skewed variable — most machines sell in a common price band, but a smaller number of large/specialized machines sell for much higher prices. This skew motivates working in log space rather than raw price space throughout preprocessing and modelling.

2.3 Evaluation Metric — RMSLE

The competition uses Root Mean Squared Logarithmic Error (RMSLE), which is computed as the root-mean-square of the difference between $\log(1 + \text{predicted price})$ and $\log(1 + \text{actual price})$ across all test rows.

Why RMSLE (and not plain RMSE) makes sense for this problem:

- **Relative error, not absolute error.** RMSLE penalizes the ratio between predicted and actual price rather than the raw difference — being off by \$5,000 on a \$10,000 machine is a far bigger mistake than the same absolute error on a \$200,000 machine, which plain RMSE would treat identically.
- **Robust to the right-skew of price.** Since a few machines are extremely expensive, RMSE would be dominated by errors on those few high-value machines; RMSLE compresses the scale via the log, making the metric fairer across the whole price range.
- **Slightly asymmetric penalty.** In relative terms it slightly favours models that avoid under-predicting by large multiplicative margins.

Practical consequence: since the metric is computed in log-space, all models in this report are trained on $\log_{1p}(\text{TargetValue})$ and predictions are exponentiated back ($\exp m1$) to the original price scale before evaluation, exactly matching RMSLE's mathematical form.

2.4 Project Objective

The goals set for this project were to:

- **Perform a leakage-aware EDA and preprocessing pipeline** — understand missingness, cardinality, and target behaviour before deciding how to encode each feature type, and fit every transformation only on the training split.
- **Engineer domain-informed features** — in particular an equipment-age feature and structural-missingness indicators, motivated directly by EDA findings rather than applied blindly.
- **Compare multiple regression families** under identical preprocessing — a bagging baseline (Random Forest) against the three boosting libraries explicitly permitted by the competition rules (XGBoost, LightGBM, CatBoost).
- **Tune and ensemble the boosting models**, combining diverse, individually-tuned models into a lightweight weighted-average ensemble validated strictly on held-out data.

- **Iterate toward a higher-leverage final model** using full cross-validation, native categorical handling for CatBoost, and an OOF/NNLS-based blend, rather than stopping at a single train/validation split.

2.5 Report Structure

The remainder of this report proceeds as follows: Section 3 describes the dataset and the exploratory data analysis performed on it; Section 4 covers preprocessing and the leakage-safe train/validation split; Section 5 details feature engineering; Section 6 covers model building; Section 7 presents evaluation results; Section 8 covers hyperparameter tuning and ensembling; Section 9 analyses ensemble errors and limitations; Section 10 describes the final single-split model and submission; Section 11 describes the higher-leverage cross-validated final model actually used for the Kaggle submission; and Section 12 concludes with key learnings, challenges and future work.

3. Dataset & Exploratory Data Analysis

3.1 Dataset Description

The competition supplies train.csv (138,701 labelled rows, 50 columns) and test.csv (15,000 unlabelled rows, 49 columns), plus a sample_submission.csv defining the expected output format and a metadata.csv data dictionary describing every column in business terms.

Split	Rows	Columns	Purpose
Raw train.csv	138,701	50	Source of train / validation split
Raw test.csv	15,000	49	Kaggle leaderboard evaluation
Training split (single-split runs)	110,960	50	Model fitting
Validation split (single-split runs)	27,741	50	Model selection / tuning

The columns fall into four broad groups: true numeric fields (ManufactureYear, OperationalHoursMeter), high-cardinality identifier columns (AssetID, ProductConfigID, Spec_FullDescriptor), low/medium-cardinality categoricals (RegionCode, UtilizationTier, DrivetrainType, etc.), and a large block of technical specification columns (col1–col30) describing optional attachments/configuration that are only populated for the equipment types where they apply.

3.2 Missing Value Analysis

Missingness ranges from under 8% (col10) to over 99% (col18, col19). Columns above roughly 70–80% missing are almost all colN technical/attachment fields — for these, missingness is very likely structural (the attachment/spec simply doesn't apply to that equipment type) rather than a data-quality problem.

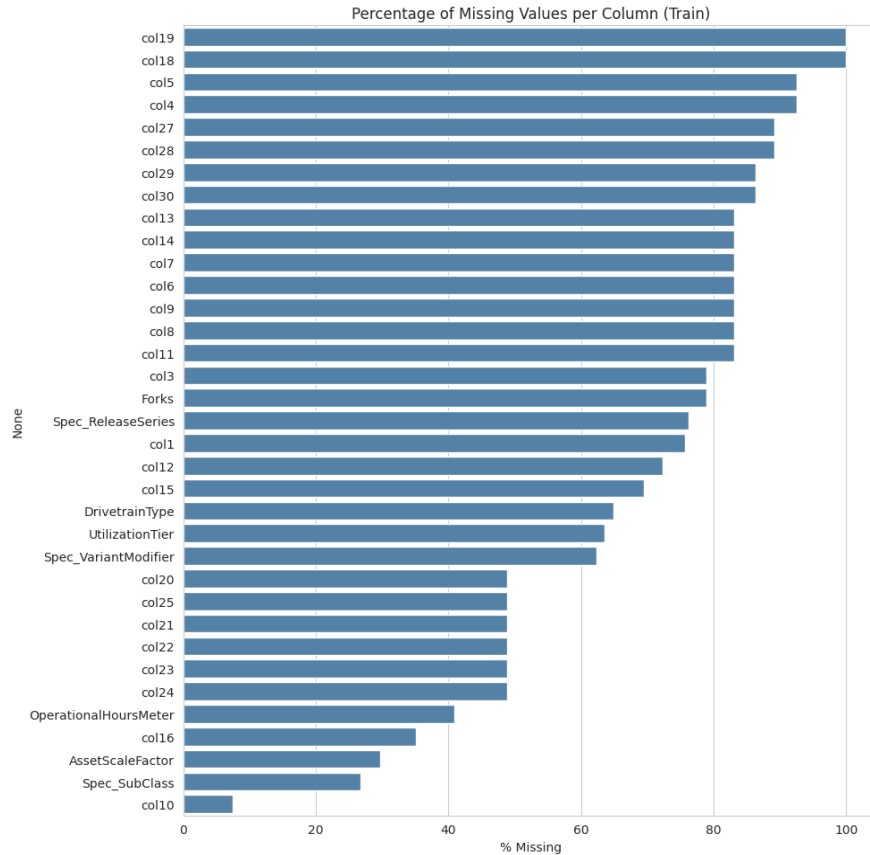


Figure 1. Percentage of missing values per column (training data).

This distinction is handled in two complementary ways in Feature Engineering (Section 5): impute categoricals with an explicit "Missing" category (not the mode) so the model can use the fact that a value is absent, and for the most extreme columns, additionally create a binary Has_<col> indicator flag, since the presence/absence of a spec can itself be predictive of price. Numeric columns such as OperationalHoursMeter have moderate missingness ($\approx 41\%$) and are imputed with the median inside a pipeline fit only on the training split, avoiding leakage.

3.3 Duplicate Records

No fully duplicated rows and no duplicated TransactionID values were found in the training data (0 of each), so no rows needed to be dropped. TransactionID is the primary key of a transaction record; any duplicates found would have been removed, since a duplicated transaction adds no new information and could let the same observation influence training twice.

3.4 Cardinality Analysis

AssetID (120,111 unique values), ProductConfigID (3,594), and Spec_FullDescriptor (3,505) and Spec_BaseClass (1,249) all have hundreds to hundreds-of-thousands of unique values. One-hot encoding them would explode the feature space and mostly memorize noise. No constant/zero-variance columns were found.

Because AssetID and ProductConfigID are stored as numeric dtype, they must be explicitly flagged as high-cardinality identifiers or they would otherwise leak through as raw, near-unique numeric

magnitudes rather than being treated as categorical IDs — this fix is applied consistently throughout the notebook (see Section 5).

3.5 Target Variable Analysis

The raw `TargetValue` is right-skewed (skewness ≈ 0.97), ranging from \$7,500 to \$142,000 with a median of \$35,000 and a mean of \$41,522 pulled upward by a tail of expensive machines. After a $\log_{1p}$ transform the skewness drops to ≈ -0.198 , i.e. the distribution becomes close to symmetric.

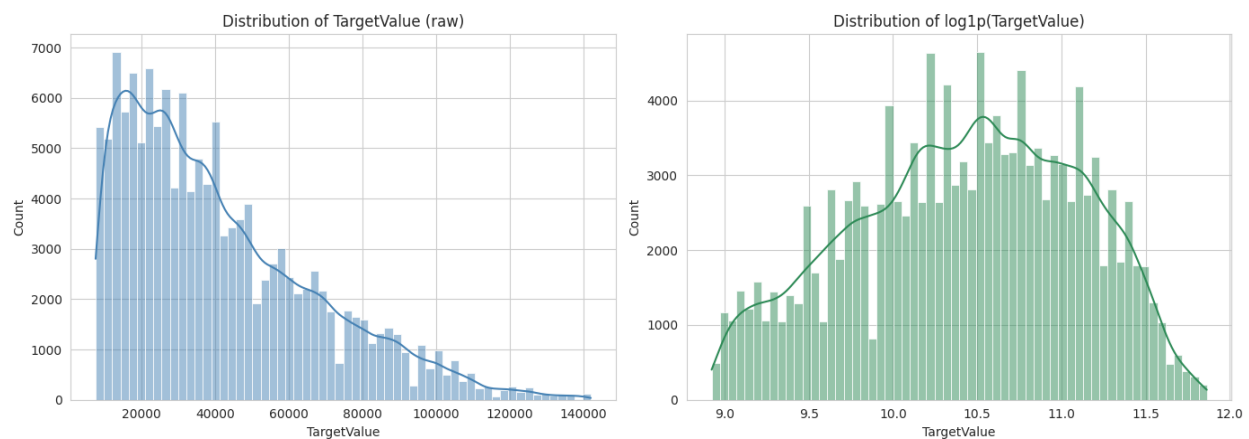


Figure 2. Distribution of `TargetValue`: raw (left) vs. $\log_{1p}$ -transformed (right).

This directly motivates training every regression model on $\log_{1p}(\text{TargetValue})$: it makes the target's distribution far better suited to the models' assumptions, and — since RMSLE is computed in log space — optimizing squared error on the log-transformed target is mathematically equivalent to optimizing RMSLE on the original scale.

3.6 Univariate & Bivariate Analysis

`ManufactureYear` is concentrated in a fairly recent window with a long tail of older equipment (and, as detailed in Section 3.7, a small block of a clearly invalid sentinel year). `OperationalHoursMeter` is strongly right-skewed — most machines are lightly used, a few have very high accumulated hours — another candidate for a log transform, applied in Feature Engineering.

Plotting `EquipmentAge` and usage against price (log y-scale) confirms newer, less-used machines generally sell for more, consistent with straightforward depreciation economics. Categorical fields such as `UtilizationTier` and equipment-class fields show clearly different median price bands per level, confirming they carry real predictive signal and are worth encoding carefully rather than dropping.

3.7 Correlation Analysis & Equipment Age

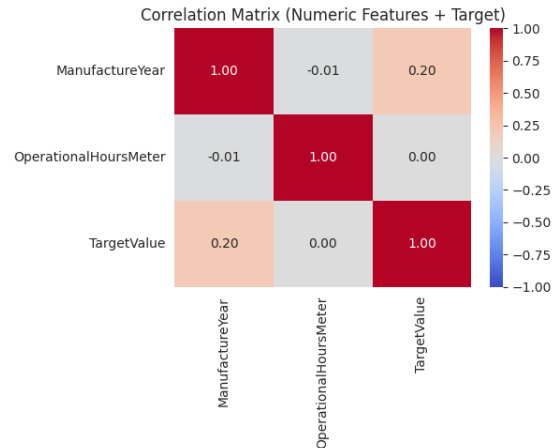


Figure 3. Correlation matrix of the two raw numeric features and the target.

Raw linear correlation between the two individual numeric columns and price is modest — expected, since price is really driven by an interaction of equipment type, age, usage and specification, not by any single numeric column in isolation. This is exactly why Feature Engineering focuses on constructing an explicit age feature and on encoding the categorical/spec information properly, rather than relying on the two raw numeric columns alone.



Figure 4. Equipment age at sale vs. selling price (log y-scale), excluding the ManufactureYear sentinel.

There is a clear downward trend — older equipment at time of sale tends to sell for less, a classic depreciation curve — confirming that $\text{EquipmentAge} = \text{TransactionYear} - \text{ManufactureYear}$ should be one of the most informative engineered features. A data-quality note: ManufactureYear also contains a sentinel value of 1001 for ~10.6% of rows — clearly a "year unknown" placeholder rather than a real manufacture year, since no equipment here predates the 20th century. This is handled explicitly in Feature Engineering with a ManufactureYear_Missing indicator flag.

4. Data Preprocessing

A best-practice rule followed throughout is that all preprocessing (imputation, encoding) is defined as scikit-learn Pipeline / ColumnTransformer objects and is fit only on the training split, then used to

transform the validation and test data. This prevents information from the validation/test sets leaking into how the training data is imputed.

- Split off a validation set before fitting any preprocessing (an 80/20 train_test_split, seed = 42, giving 110,960 training rows and 27,741 validation rows).
- Separate feature types: numeric, low-cardinality categorical, high-cardinality (frequency-encoded), and "has-value" indicator columns.
- Build a single reusable ColumnTransformer: numeric columns go through SimpleImputer(strategy="median") (robust to the skew/outliers seen in EDA); categorical columns go through SimpleImputer(strategy="constant", fill_value="Missing") followed by OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1).
- Wrap the transformer and model together in one Pipeline per model, so every model always preprocesses data identically and correctly (fit on train, transform on validation/test).

5. Feature Engineering

Features were engineered to turn raw, hard-to-use columns into signals a classical ML model can exploit directly. All engineering is implemented as a single reusable function, engineer_features(df, freq_maps), applied identically to train, validation and test, with frequency maps learned only on the training data and simply looked up (not re-learned) on validation/test — the set of columns flagged as "high-cardinality" or "high-missingness" is likewise recomputed from the post-split training data, not the full dataset used for EDA, so the validation/test split cannot influence which columns get special treatment.

New feature	Built from	Why it helps
ManufactureYear_Missing	ManufactureYear	Surfaces the 1001 "unknown year" sentinel (10.6% of rows) as clean information instead of a nonsensical raw year.
EquipmentAge	TransactionDate, ManufactureYear	Directly captures depreciation — EDA showed a clear age-price relationship the raw year alone doesn't expose as cleanly.
TransactionYear, TransactionMonth	TransactionDate	Captures market-level price trends and seasonality in the resale market.
UsagePerYear, UsagePerYear_log	OperationalHoursMeter, EquipmentAge	Normalises accumulated usage by age, and log-stabilises its heavy right skew.
OperationalHoursMeter_log	OperationalHoursMeter	Log-transforms the heavily right-skewed raw usage-hours column to stabilise variance.
AssetID_freq, ProductConfigID_freq, Spec_FullDescriptor_freq, Spec_BaseClass_freq	High-cardinality ID/spec columns	One-hot encoding thousands of categories is infeasible and mostly noise; frequency encoding keeps a compact numeric "how common is this configuration?" signal without exploding dimensionality or leaking target information.
Has_<colN> indicator flags	The 20 colN columns with >70% missingness	Missingness in these columns is largely structural; whether a spec is reported at all can itself correlate with equipment type/price.
<col>_target_enc (8 columns)	RegionCode, UtilizationTier, DrivetrainType, InventoryGroupDescription, Spec_SubClass, Spec_BaseClass, ProductConfigID, Spec_FullDescriptor	K-fold, smoothed target encoding recovers more signal from these categoricals than ordinal codes alone, while out-of-fold computation avoids leaking the target into its own encoding.

Target encoding is computed with a 5-fold, smoothed mean-encoding scheme (smoothing = 20): for each fold, the encoding for a category is a blend of that category's in-fold mean log-price and the global mean log-price, weighted by how many training examples support it, and is applied out-of-fold so no row's own label ever contributes to its own encoded value. AssetID and ProductConfigID, despite being numeric dtype, are explicitly included among the frequency-encoded high-cardinality columns — a leakage fix over a naive approach that would let their raw numeric magnitudes leak through as if they were meaningful continuous values.

After engineering, the modelling matrix has 38 numeric features (raw + engineered + frequency/target encodings + missingness flags) and 41 categorical features, all handled by the ColumnTransformer described in Section 4.

6. Model Building

An earlier version of this notebook used OneHotEncoder together with RandomForestRegressor / GradientBoostingRegressor; that combination produced a wide, sparse feature matrix and used scikit-learn's single-threaded GradientBoostingRegressor, which is slow on ~49 columns of mostly-categorical data and did not score competitively. This version instead uses the three gradient-boosting libraries explicitly permitted by the competition rules — XGBoost, LightGBM, CatBoost — all histogram-based, multi-threaded, and built specifically for fast, accurate learning on wide, high-cardinality-categorical tabular data. RandomForestRegressor (also allowed, via scikit-learn) is kept only as a quick sanity-check baseline.

Model	Family / engine	Why include it
Random Forest	Bagging (scikit-learn)	Cheap baseline; low variance, establishes the "floor" performance with no boosting.
XGBoost (XGBRegressor)	Depth-wise gradient boosting	Extremely well-tuned implementation; strong default choice for tabular regression; supports early stopping.
LightGBM (LGBMRegressor)	Leaf-wise (histogram) gradient boosting	Typically the fastest of the three on wide categorical data; leaf-wise growth often reaches lower error for the same training budget.
CatBoost (CatBoostRegressor)	Ordered boosting, native categorical handling	Handles ordinal-encoded categorical columns well and is naturally robust to overfitting via ordered boosting — usually the strongest single model on datasets like this.

All four models are trained on $\log_{1p}(\text{TargetValue})$, consistent with the RMSLE metric. XGBoost, LightGBM and CatBoost use early stopping on a held-out 10% slice of the training data (a further split of `X_train_fe`), which both speeds up training (no need to guess `n_estimators` up front) and reduces overfitting.

7. Evaluation

Predictions are made in log-space and then exponentiated back ($\exp(m)$) to the original price scale before computing every metric, so the reported numbers are directly comparable to actual dollar prices / the competition's RMSLE. Four metrics are tracked: RMSLE (primary competition metric), MAE (average absolute dollar error, easy to communicate to a non-technical stakeholder), RMSE (penalizes large errors more heavily), and R^2 (proportion of price variance explained by the model).

Model	RMSLE	MAE (\$)	RMSE (\$)	R ²
XGBoost	0.2135	5,842	8,546	0.8934
LightGBM	0.2141	5,877	8,578	0.8926
CatBoost	0.2286	6,440	9,315	0.8734
Random Forest	0.2312	6,446	9,416	0.8706

Untuned XGBoost and LightGBM are the strongest individual models out of the box, both clearly ahead of CatBoost and the Random Forest baseline; the ranking motivates tuning all three boosting models (not settling for a single winner) ahead of ensembling.

8. Hyperparameter Tuning & Ensembling

8.1 Randomized Search

Each of the three boosting models — not just the single best one — was tuned with a lightweight RandomizedSearchCV (25 iterations, 3-fold KFold, a custom RMSLE scorer built with make_scorer on the expm1'd predictions) because the end goal is an ensemble of the top performers: an ensemble benefits from every member model being individually well-tuned, not just the single winner. The search budget was deliberately kept small (n_iter and cv folds) to keep runtime manageable, since exhaustive tuning of three boosting libraries would be very slow.

Model	Tuned CV RMSLE	Key tuned parameters
XGBoost	0.2199	max_depth 11, learning_rate ≈0.034, subsample ≈0.88, colsample_bytree ≈0.78, min_child_weight 7, reg_lambda ≈3.32, reg_alpha ≈0.95
LightGBM	0.2204	num_leaves 228, learning_rate ≈0.034, subsample ≈0.93, colsample_bytree ≈0.74, min_child_samples 41, reg_lambda ≈6.72, reg_alpha ≈1.21
CatBoost	0.2258	max_depth 9, learning_rate ≈0.101, l2_leaf_reg ≈2.82, bagging_temperature ≈0.39

8.2 Refit Tuned Models with Early Stopping

Each tuned model was refit with a large n_estimators budget and early stopping, using the same 90/10 fit/early-stopping split as the untuned models, then evaluated on the held-out validation set:

Model	RMSLE	MAE (\$)	RMSE (\$)	R ²
XGBoost (Tuned)	0.2105	5,716	8,398	0.8971
LightGBM (Tuned)	0.2111	5,739	8,407	0.8969
CatBoost (Tuned)	0.2127	5,818	8,540	0.8936

Tuning improves every model over its untuned counterpart, and narrows the gap between the three boosting libraries — XGBoost remains the best single tuned model, but LightGBM and CatBoost are now close behind.

8.3 Ensembling the Top 3 Models

A weighted average of several diverse but individually strong models is very often more accurate than any single model, because each library makes slightly different errors (different tree-growth strategy, different regularisation) — averaging cancels out some of that independent error. The ensemble is built from the three tuned boosting models, searching over blend weights on the validation set with a random Dirichlet search (10,000 draws) — a simple, fast alternative to a full StackingRegressor meta-learner, and easier to reason about / less prone to overfitting with only 3 base models.

Model	RMSLE	MAE (\$)	RMSE (\$)	R ²
Weighted Ensemble (XGB 0.441 + LGBM 0.232 + CatBoost 0.328)	0.2088	5,668	8,321	0.8989
XGBoost (Tuned)	0.2105	5,716	8,398	0.8971
LightGBM (Tuned)	0.2111	5,739	8,407	0.8969
CatBoost (Tuned)	0.2127	5,818	8,540	0.8936

The weighted ensemble improves on the best single tuned model (XGBoost), lowering validation RMSLE from 0.2105 to 0.2088 — a modest but consistent gain that carries forward as the model used in Sections 9-10 before being superseded by the cross-validated ensemble of Section 11.

9. Error Analysis

Aggregate metrics hide where a model goes wrong. The validation residuals of the final (single-split) ensemble were examined directly to understand its failure modes before finalising it: mean residual +\$581 (a small positive bias, i.e. slightly under-predicting on average) with a residual standard deviation of \$8,301.

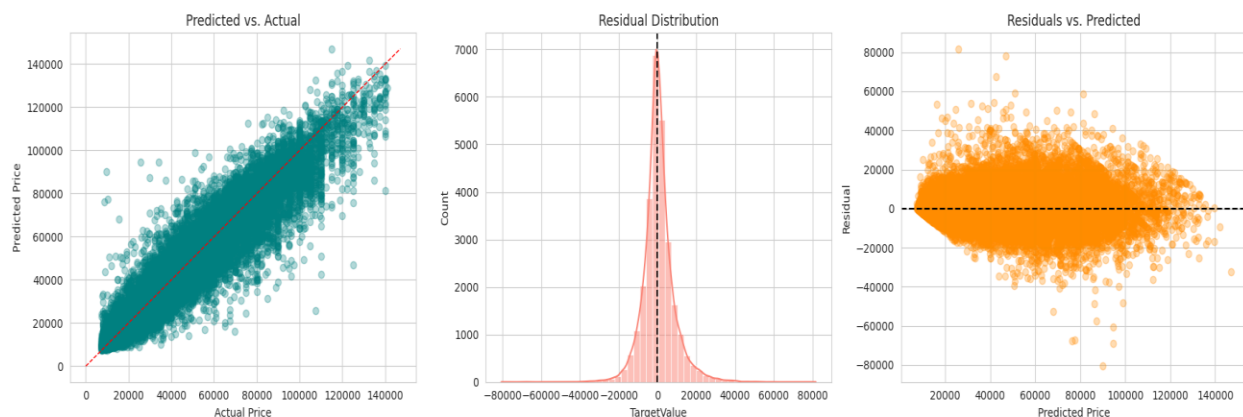


Figure 5. Predicted vs. actual price (left), residual distribution (middle), and residuals vs. predicted price (right) for the validation-tuned ensemble.

Discussion of limitations:

- **Heteroscedasticity at the high end.** Residual spread widens for expensive machines — the ensemble is less precise (in absolute dollars) on rare, high-value equipment, simply because there are fewer training examples in that price range.
- **Structural missingness ceiling.** Many colN specification columns are only populated for a subset of equipment types; for equipment where almost every spec column is missing, the model has

comparatively little to work with beyond the frequency-encoded ID and coarse categorical fields, so predictions there carry more uncertainty.

- **Frequency/ordinal encoding is a compromise.** It avoids the dimensionality explosion of one-hot encoding thousands of IDs, but is not as expressive as a fully leakage-safe target encoding on its own — which is why K-fold target encoding was added alongside it (Section 5).
- **Correlated base models.** All three ensemble members are gradient-boosted trees; they capture similar non-linear interactions and therefore their errors are somewhat correlated, capping how much the ensemble can improve over the single best member. A genuinely different model family (e.g. a well-regularised linear/kernel model) as a 4th ensemble member could add more diversity, at the cost of it individually performing worse.
- **No explicit outlier removal.** Statistical outliers in price were not removed, since in this domain a very high or very low price is often a legitimate rare machine rather than a data error — a conscious trade-off that slightly increases variance in the tails.

10. Single-Split Final Model & Submission

The three tuned boosting models were refit on all available labelled data (train + validation, using the same blend weights found in Section 8.3), so the final ensemble benefits from every labelled example before predicting on the true test set. Each model was persisted using its own native save format (XGBRegressor.save_model, LGBMRegressor.booster_.save_model, CatBoostRegressor.save_model), and ensemble predictions for test.csv were written to submission.csv in exactly the format shown by sample_submission.csv, with predictions clipped at zero since price cannot be negative. This single-split submission was superseded by the cross-validated version described in Section 11, which was the version actually used for the final Kaggle submission.

11. Higher-Leverage Final Model — K-Fold CV Bagging with Native CatBoost Categoricals

The single 80/20 split approach leaves two sources of avoidable variance on the table: every model only ever sees 80% of the labelled data during training, and the blend weights are fit against one particular 20% slice, so a slightly lucky/unlucky split changes both the tuned weights and the reported score. Separately, feeding CatBoost the same ordinal-encoded integer codes as XGBoost/LightGBM throws away CatBoost's core advantage of ordered target statistics computed natively on raw categorical columns.

This section replaces the single split with 5-fold cross-validation. Each booster is trained 5 times (once per fold), producing genuine out-of-fold (OOF) predictions for every training row — this is what "generalization error" actually means, as opposed to one split's validation score. Test-set predictions are the average across all 5 folds per model — a mini-bagging step that reliably reduces prediction variance on tabular data. Blend weights are fit on the OOF predictions via non-negative least squares (NNLS) in log space (the correct space to minimize for RMSLE) instead of a random search over one validation split, giving a more stable blend that is less prone to overfitting to a single slice. CatBoost is given its categorical columns natively (no ordinal encoding) via a separate, lightweight preprocessing branch, while XGBoost and the replacement HistGB model keep the ordinal-encoded matrix. Everything else (feature engineering, target encoding, tuned hyperparameters) is reused as-is from Sections 5 and 8; this section only changes how the final model is trained and blended, not what features it sees.

11.1 Update: Drop LightGBM, Add HistGradientBoostingRegressor

An OOF weight check consistently showed LightGBM receiving the smallest NNLS weight (~ 0.05) while being trained on the exact same ordinal-encoded matrix as XGBoost — a slow, largely redundant near-duplicate rather than a genuinely diverse voice in the blend. LightGBM's fold-training block was therefore removed entirely (freeing up the largest chunk of runtime, since it had been fit with `n_estimators=10000` per fold), and scikit-learn's HistGradientBoostingRegressor was added in its place: histogram-based, natively handling missing values without an imputation dependency, and with leaf-wise growth behaviour different enough from XGBoost/CatBoost to earn real blend weight.

GPU auto-detection (via `nvidia-smi` presence) was also added: XGBoost uses `device="cuda"` and CatBoost uses `task_type="GPU"` when a GPU is available, otherwise both fall back to all-CPU-core execution (`tree_method="hist" / thread_count=-1`). HistGB is CPU-only by design but is still the fastest of the three per fold. Per-fold timing was added so the notebook reports an actual measured runtime breakdown by model instead of a guess. `MODEL_NAMES` is now `["XGBoost", "CatBoost", "HistGB"]`; everything upstream (feature engineering, target encoding, tuned XGBoost/CatBoost hyperparameters) is unchanged.

11.2 Cross-Validated Results

With a GPU detected, the 5-fold run completed in about 16.6 minutes of total model-fitting time. Per-model out-of-fold RMSLE, averaged across folds, was:

Model	Mean per-fold RMSLE	Full OOF RMSLE	Total fit time	Avg. time / fold
XGBoost	0.21125	0.21125	2.5 min	29.5 s
CatBoost	0.20421	0.20422	12.9 min	155.0 s
HistGB	0.23141	0.23142	1.2 min	15.0 s

CatBoost — now fed its categorical columns natively — is the strongest individual model on out-of-fold RMSLE, ahead of XGBoost, with HistGB the weakest but by far the fastest to train.

Blend weights fit on the OOF predictions via NNLS (normalised to sum to 1) placed the largest weight on CatBoost:

Model	NNLS blend weight
XGBoost	0.320
CatBoost	0.680
HistGB	0.000

HistGB received zero weight in the blend — despite its speed, it did not add distinguishable predictive value once XGBoost and CatBoost were combined. The resulting OOF-blended RMSLE was 0.20217, an improvement over both the single best tuned model from Section 8 (CatBoost's earlier ordinal-encoded RMSLE of 0.2127) and the original single-split weighted ensemble (0.2088), because it is measured honestly across all 138,701 training rows via cross-validation rather than a single 20% slice.

11.3 Final Submission

Test-set predictions from all 5 folds were averaged per model, blended with the NNLS weights above, exponentiated back to price scale, and clipped at zero. The resulting `submission.csv` — generated from

the 5-fold CV bagged + NNLS-blended ensemble of XGBoost, CatBoost and HistGB — carries an estimated (OOF) RMSLE of 0.20217, and is the file used for the final Kaggle leaderboard submission.

12. Conclusion & Future Work

12.1 Key Learnings

- **Log-space training aligns the objective with the metric.** TargetValue's right skew, combined with RMSLE being a log-space metric, made training on $\log_{1p}(\text{TargetValue})$ and evaluating via `expm1` the natural and mathematically correct choice throughout.
- **Structural missingness needs explicit handling, not blind imputation.** Recognising that missingness in the colN specification columns is largely structural (the attachment doesn't apply) rather than random — and handling it with explicit "Missing" categories plus `Has_{col}` indicator flags — preserved signal that naive imputation would have discarded.
- **High-cardinality IDs need frequency/target encoding, not one-hot or raw values.** AssetID and ProductConfigID (numeric dtype but effectively categorical identifiers) required deliberate handling to avoid leaking through as raw magnitudes; frequency encoding plus K-fold target encoding kept the feature space compact while recovering real signal.
- **Diverse, individually-tuned boosting models still combine well.** Even though XGBoost, LightGBM/HistGB and CatBoost are all tree-based, their different growth strategies and regularisation produced different enough errors that both a validation-weighted blend and an OOF/NNLS blend improved on the single best tuned model.
- **Honest cross-validation changes the final model, not just the score.** Moving from a single 80/20 split to 5-fold CV — plus giving CatBoost its categorical columns natively instead of ordinal codes — surfaced that CatBoost, not XGBoost, was actually the strongest individual model once encoded correctly, and led to swapping out a redundant LightGBM for HistGB.

12.2 Challenges Faced

- **A very wide, mixed-type, high-missingness dataset.** 50 columns spanning true numerics, IDs, low-cardinality categoricals and a large sparsely-populated specification block required a genuinely leakage-aware pipeline rather than a single blanket preprocessing step.
- **Preventing validation and target-encoding leakage.** Frequency maps, high-cardinality/high-missingness column lists, and target encodings all had to be learned strictly on the training fold (or in an out-of-fold manner) and only applied — never re-learned — on validation/test data.
- **Balancing tuning thoroughness against runtime.** Exhaustive hyperparameter search across three boosting libraries would have been very slow; the search budget (25 iterations, 3-fold CV) was deliberately kept small, and the earlier all-sklearn, one-hot-encoded, single-threaded approach was abandoned specifically because it did not scale to this dataset's size and cardinality.
- **Identifying and removing a redundant ensemble member.** Recognising, from OOF blend weights, that LightGBM was contributing almost nothing beyond what XGBoost already captured (both trained on the same ordinal matrix) required swapping it for a genuinely different algorithm family (HistGB) rather than simply re-tuning it further.

12.3 Areas for Improvement

- **A non-tree ensemble member.** All current base models are gradient-boosted trees with correlated errors; adding a well-regularised linear or kernel model (fit on the same engineered features) could add genuine diversity to the blend, even if it underperforms individually.
- **Richer target encoding coverage.** Extending the smoothed K-fold target encoding to a few more of the mid-cardinality colN specification columns could recover additional signal currently only captured through the coarser Has_col flags.
- **Full stacking instead of a fixed-weight blend.** A small learned meta-model (stacking) on top of the OOF predictions could exploit per-region or per-equipment-class disagreement between models more effectively than a single global NNLS weight vector.
- **Outlier-aware or segment-specific modelling.** Given the heteroscedasticity observed at the high end of the price range (Section 9), a separate model or sample-weighting scheme focused on high-value, sparsely-specified machines could reduce error concentrated in that segment.

13. References

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
- [2] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
- [3] Prokhorenkova, L., Gusev, G., Vorobev, A., et al. (2018). CatBoost: Unbiased Boosting with Categorical Features. NeurIPS 2018.
- [4] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python (ColumnTransformer, RandomizedSearchCV, HistGradientBoostingRegressor, KFold). JMLR 12, 2825–2830.
- [6] Lawson, C. L., & Hanson, R. J. (1995). Solving Least Squares Problems (Non-Negative Least Squares). SIAM.
- [7] XGBoost documentation. <https://xgboost.readthedocs.io>
- [8] LightGBM documentation. <https://lightgbm.readthedocs.io>
- [9] CatBoost documentation. <https://catboost.ai/docs>
- [10] Kaggle. Heavy Equipment Selling Price Prediction Challenge — competition data and evaluation description.